

P6. Ejercicio de la Paradoja de Dios.

Garay Pérez Matthew Ilveneff

202210036@tesco.edu.mx

Tecnológico de Estudios Superiores
de Coacalco

Ingeniería en Sistemas
Computacionales

Inteligencia Artificial

Ing. Vivanco Benavides Luis Enrique

Entrega 13 de octubre de 2025

3821

Introducción

El lenguaje Prolog (Programming in Logic) se fundamenta en la programación declarativa, donde los problemas se describen mediante hechos, reglas y consultas, y no a través de instrucciones secuenciales. En esta práctica, se explora el uso de estructuras lógicas y la manipulación de listas mediante los predicados `member/2` y `append/3`, así como la representación de un razonamiento filosófico sobre la existencia del mal.

El predicado `member/2` permite determinar si un elemento pertenece a una lista, lo cual es esencial en la manipulación de datos en Prolog, ya que las listas constituyen una de sus estructuras más básicas y potentes. Por otro lado, el predicado `append/3` se utiliza para unir o concatenar listas, formando nuevas estructuras de datos a partir de listas existentes. Ambos comandos son herramientas clave para comprender la recursividad y la unificación, dos conceptos fundamentales en la programación lógica (Bratko, 2012).

Además, la práctica integra un ejercicio lógico basado en el problema teológico del mal, representado también en un diagrama. Este dilema, de raíz filosófica, cuestiona la compatibilidad entre la existencia del mal y los atributos de un ser divino omnipotente, omnisciente y bondadoso. En este contexto, Prolog se convierte en un medio idóneo para representar razonamientos complejos y cadenas de decisiones, simulando el proceso de pensamiento humano mediante reglas condicionales (Russell & Norvig, 2021).



Desarrollo

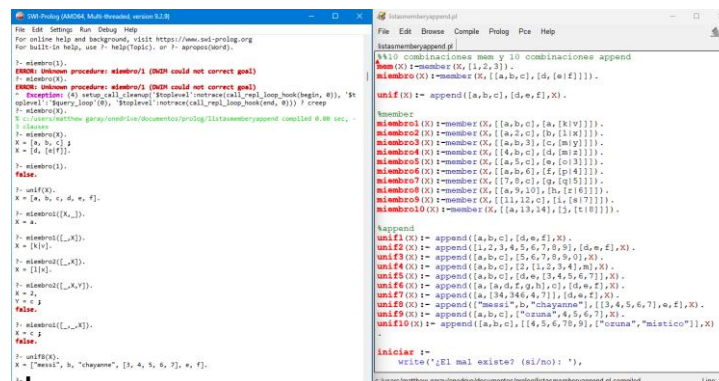
. Esta práctica se compone de tres partes principales, orientadas al desarrollo del razonamiento lógico y la manipulación de estructuras de datos en Prolog. Cada una de ellas aporta un aprendizaje distinto: desde el manejo básico de listas hasta la representación simbólica de un dilema filosófico.

1. Uso de las listas member/2

En la primera parte se realizaron diez combinaciones con el predicado member/2, que se utiliza para determinar si un elemento pertenece a una lista. Este predicado puede emplearse tanto para consultar si un valor está dentro de una lista como para generar elementos a partir de la misma. Por ejemplo, member(X, [1,2,3]) permite que Prolog devuelva los valores 1, 2 y 3 sucesivamente.

En esta práctica se utilizaron ejemplos más complejos, incluyendo listas anidadas y listas con cabeza y cola, lo que permite comprender mejor cómo Prolog maneja estructuras jerárquicas. Algunos ejemplos, como [[a,b,c],[d,[e,f]]], ilustran cómo Prolog puede trabajar con listas dentro de listas, e incluso con variables que representan partes desconocidas de una lista. Esto refuerza el concepto de unificación, donde el lenguaje compara estructuras y sustituye variables hasta que las expresiones sean equivalentes.

De esta forma, la sección de member no solo permite verificar pertenencias, sino también analizar el modo en que Prolog busca soluciones en su base de conocimientos, demostrando el poder del razonamiento lógico mediante búsqueda y coincidencia de patrones.



```

%10 combinaciones mem y 10 combinaciones append
member(X, [1,2,3]).
member(X) :- member(X, [a,b,c], [d,[e,f]]).
unif(X) :- append([a,b,c], [d,e,f], X).

member(X) :- member(X, [a,b,c], [a,[b,c]]).
member(X) :- member(X, [a,b,c], [b,[a,c]]).
member(X) :- member(X, [a,b,c], [c,[a,b]]).
member(X) :- member(X, [a,b,c], [d,[a,c]]).
member(X) :- member(X, [a,b,c], [e,[a,b]]).
member(X) :- member(X, [a,b,c], [f,[a,b]]).
member(X) :- member(X, [a,b,c], [g,[a,b]]).
member(X) :- member(X, [a,b,c], [h,[a,b]]).
member(X) :- member(X, [a,b,c], [i,[a,b]]).
member(X) :- member(X, [a,b,c], [j,[a,b]]).

append([a,b,c], [d,e,f], X).
unif(X) :- append([a,b,c], [d,e,f], X).
unif(X) :- append([a,b,c], [d,e,f], X).
unif(X) :- append([a,b,c], [d,e,f], X).
unif(X) :- append([a,b,c], [d,e,f], X).
unif(X) :- append([a,b,c], [d,e,f], X).
unif(X) :- append([a,b,c], [d,e,f], X).
unif(X) :- append([a,b,c], [d,e,f], X).
unif(X) :- append([a,b,c], [d,e,f], X).
unif(X) :- append([a,b,c], [d,e,f], X).

iniciar :-
write('¿El mal existe? (si/no): '),

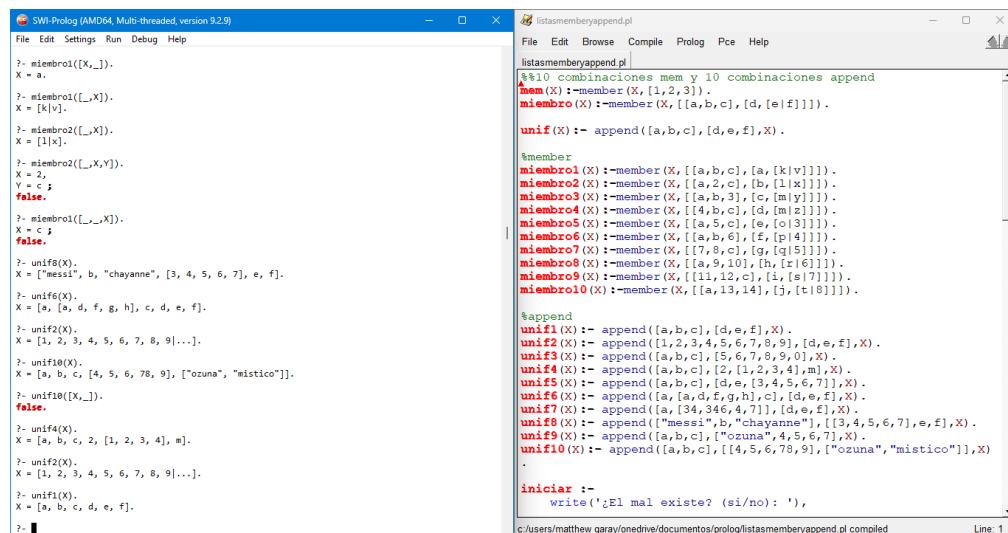
```

2. Uso de las listas append/3

En la segunda sección se realizaron diez combinaciones con el predicado append/3, el cual une dos listas y produce una tercera como resultado. Este predicado puede verse como una herramienta fundamental para construir, extender o dividir listas, algo indispensable en la programación declarativa.

Por ejemplo, al ejecutar `append([a,b,c],[d,e,f],X)`, el programa genera la lista resultante `X = [a,b,c,d,e,f]`. En los ejemplos más complejos, se incluyeron listas internas y elementos mixtos (átomos, números y cadenas), como en `append(["messi",b,"chayanne"],[[3,4,5,6,7],e,f],X)`. Esto demuestra que Prolog puede manipular información heterogénea sin necesidad de especificar tipos de datos, pues el motor de inferencia se encarga de realizar la unificación de manera automática.

El aprendizaje principal de esta parte radica en la comprensión de cómo Prolog realiza operaciones recursivas sobre listas. Detrás del funcionamiento de `append/3` se encuentra un proceso de descomposición: se separa la cabeza de la lista y se añade elemento por elemento hasta formar la nueva lista. Este comportamiento refleja la naturaleza declarativa del lenguaje, que no “ordena” realizar una acción, sino que “describe” la relación lógica que debe cumplirse entre las listas.



```

SWI-Prolog (AMD64, Multi-threaded, version 9.2.9)
File Edit Settings Run Debug Help
?- miembro1([X,_]).
X = a.
?- miembro1([_,X]).
X = [k|v].
?- miembro2([_,X]).
X = [1|x].
?- miembro2([_,X,Y]).
X = 2,
Y = c ;
false.
?- miembro1([_,_,X]).
X = c ;
false.
?- unifi8(X).
X = ["messi", b, "chayanne", [3, 4, 5, 6, 7], e, f].
?- unifi6(X).
X = [a, [a, d, f, g, h], c, d, e, f].
?- unifi2(X).
X = [1, 2, 3, 4, 5, 6, 7, 8, 9|...].
?- unifi0(X).
X = [a, b, c, [4, 5, 6, 78, 9], ["ozuna", "mistico"]].
?- unifi0(X,_).
false.
?- unifi4(X).
X = [a, b, c, 2, [1, 2, 3, 4], #].
?- unifi2(X).
X = [1, 2, 3, 4, 5, 6, 7, 8, 9|...].
?- unifi1(X).
X = [a, b, c, d, e, f].
?-

listamembriyappend.pl
File Edit Browse Compile Prolog Pce Help
listamembriyappend.pl
%%10 combinaciones mem y 10 combinaciones append
%mem(X):-member(X,[1,2,3]).
miembro(X):-member(X,[a,b,c],[d,[e|f]]).
unif(X):-append([a,b,c],[d,e,f],X).
$member
miembro1(X):-member(X,[a,b,c],[a,[k|v]]).
miembro2(X):-member(X,[a,2,c],[b,[1|x]]).
miembro3(X):-member(X,[a,b,3],[c,[m|y]]).
miembro4(X):-member(X,[4,b,c],[d,[m|z]]).
miembro5(X):-member(X,[a,5,c],[e,[o|3]]).
miembro6(X):-member(X,[a,b,6],[f,[p|4]]).
miembro7(X):-member(X,[7,8,c],[g,[q|5]]).
miembro8(X):-member(X,[a,9,10],[h,[r|6]]).
miembro9(X):-member(X,[11,12,c],[i,[s|7]]).
miembro10(X):-member(X,[a,13,14],[j,[t|8]]).
$append
unifi1(X):-append([a,b,c],[d,e,f],X).
unifi2(X):-append([1,2,3,4,5,6,7,8,9],[d,e,f],X).
unifi3(X):-append([a,b,c],[5,6,7,8,9,0],X).
unifi4(X):-append([a,b,c],[2,[1,2,3,4],m],X).
unifi5(X):-append([a,b,c],[d,e,[3,4,5,6,7]],X).
unifi6(X):-append([a,[a,d,f,g,h],c],[d,e,f],X).
unifi7(X):-append([a,[34,346,4,7]],d,e,f,X).
unifi8(X):-append(["messi",b,"chayanne"],[[3,4,5,6,7],e,f],X).
unifi9(X):-append([a,b,c],["ozuna",4,5,6,7],X).
unifi10(X):-append([a,b,c],[4,5,6,78,9],["ozuna","mistico"],X).
.
iniciar :-
write('¿El mal existe? (si/no): '),

```

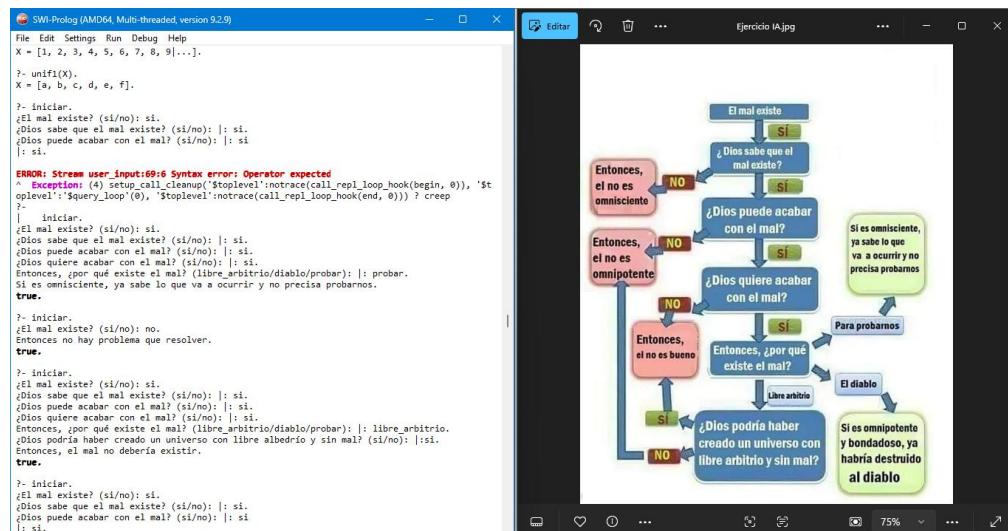
3. Representación lógica de la paradoja de Dios

La tercera parte de la práctica representa el razonamiento lógico sobre la existencia del mal, inspirada en el diagrama adjunto. Este programa es un excelente ejemplo de cómo Prolog puede modelar razonamientos complejos o dilemas éticos a través de reglas y condiciones.

El código inicia preguntando al usuario “¿El mal existe?”, y según la respuesta, continúa evaluando los atributos divinos. Si se responde que el mal no existe, el programa concluye que no hay problema que resolver. En cambio, si se responde que sí, se inicia una secuencia de preguntas sobre si Dios sabe que el mal existe (omnisciencia), si puede acabar con él (omnipotencia) y si quiere hacerlo (bondad).

Dependiendo de las respuestas, el programa muestra diferentes conclusiones, como “Entonces, Él no es omnipotente” o “Entonces, Él no es bueno”. Además, en caso de aceptar que Dios es omnipotente y bueno, el programa introduce explicaciones posibles como “libre albedrío”, “el diablo” o “probar al ser humano”.

Esta sección combina razonamiento filosófico con lógica formal, mostrando cómo Prolog puede usarse para representar árboles de decisión, modelar discusiones éticas o incluso simular conversaciones racionales. En términos de programación lógica, cada rama del diálogo es un predicado que se activa según la respuesta del usuario, demostrando el flujo natural del razonamiento deductivo.



Conclusiones

El desarrollo de esta práctica permitió comprender la versatilidad del lenguaje Prolog como herramienta tanto para la representación del conocimiento como para la manipulación de estructuras de datos. En las primeras dos partes, el uso de los predicados `member/2` y `append/3` mostró cómo Prolog maneja listas de manera lógica y recursiva, lo que resulta esencial para tareas como la búsqueda, la combinación de elementos y la generación de nuevas estructuras.

El análisis de los resultados permitió observar cómo la unificación y la recursividad son pilares de este lenguaje, ya que permiten resolver consultas complejas sin necesidad de bucles ni estructuras de control imperativas. Este enfoque lógico fortalece el pensamiento computacional, al exigir un razonamiento abstracto basado en relaciones y no en procedimientos.

En cuanto al razonamiento sobre la existencia del mal, la práctica demostró que Prolog puede representar problemas filosóficos o éticos mediante lógica simbólica. El programa, basado en el diagrama, actúa como un sistema experto básico que guía al usuario a través de un conjunto de preguntas y conclusiones, ejemplificando el funcionamiento de un razonador lógico.

De manera general, esta práctica no solo permitió aplicar conocimientos técnicos de Prolog, sino también comprender cómo la lógica formal puede emplearse para modelar el pensamiento humano, la toma de decisiones y los dilemas morales. Esto refuerza la idea de que la inteligencia artificial no solo abarca cálculos matemáticos, sino también la representación simbólica del conocimiento y el razonamiento inteligente.

En conclusión, el ejercicio integró con éxito la manipulación de listas, la lógica declarativa y el razonamiento deductivo en un mismo entorno, consolidando así la comprensión de los principios fundamentales de la programación en Prolog y su relevancia dentro de la inteligencia artificial moderna.

Referencias bibliográficas

Bratko, I. (2012). Prolog Programming for Artificial Intelligence (4th ed.). Pearson Education.

Russell, S. J., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

Nilsson, N. J. (2014). Principles of Artificial Intelligence. Morgan Kaufmann.